José Rizal University

First Semester, Academic Year 2026 - 2027

PhytoQuest: A Location-Aware Botanical Research Support System

In partial fulfillment of the requirements for the course ITC C301-301I – Integrative Programming and Technologies 1

Submitted by:

Leader:

Dival, Francesca Isabel

Members:

Aguilar, Trisha Nicole

Mondares, Matt

Speirs, Keith Alan

Submitted to:

Dr. Richard Monreal

ITC C301-301I Professor

PhytoQuest: A Location-Aware Botanical Research Support System

***Abstract*—**PhytoQuest is a responsive web application that supports the early fieldwork stage of botanical and pharmaceutical research. It combines plant search, GPS-confirmed sightings, image-based species identification, and source-backed plant profiles in a single research tool. The system is designed for pharmacy researchers, botanists, medical or biomedical engineers, students, and qualified reviewers. Its database records botanical traits, habitat, location evidence, known toxins, documented extract types, and research references. It does not diagnose, prescribe, recommend dosage, or advise plant consumption. React JS provides the client interface; Laravel PHP provides the application and data services; and Python provides a machine-learning identification service. Sensitive records receive controlled map visibility to protect private property and vulnerable plant populations. The first release is limited to a pilot campus or selected partner locations so submissions can be reviewed and evaluated responsibly.

Index Terms*—botanical informatics, computer vision, geospatial systems, medicinal plants, research support, species occurrence data.*

I. INTRODUCTION

Early botanical and pharmaceutical studies often spend substantial time locating specimens, recording field observations, and consolidating introductory information before laboratory work can begin. Existing online sources often contain useful plant information but are commonly separated from local occurrence evidence and practical field workflows. PhytoQuest addresses this gap by joining a searchable research profile with a reviewed sighting map and an image-identification workflow.

The system treats a mobile GPS reading as evidence supplied by the user, not as proof that a plant is present. A submitted sighting therefore includes coordinates, date, images, plant part tags, location ownership status, and notes. It becomes a trusted record only after the selected verification rules are satisfied. This distinction keeps the map useful without presenting unverified observations as facts.

Interest in event-driven design has grown alongside the broader shift toward real-time, asynchronous, and microservice-based web systems, where user interfaces are expected to reflect background work — search results, uploads, model inference, moderation outcomes — as it completes, rather than only after a full page reload. Adoption of the specific mechanisms that deliver this, however, is uneven in practice: a large-scale crawl of the top one million websites found that HTTP polling remains far more common than WebSocket-based push delivery, and that WebSocket adoption itself has stagnated in recent years even as demand for real-time features has grown [16]. This gap between rising demand for real-time behavior and comparatively slow adoption of the event-driven mechanisms that provide it is the industry context PhytoQuest sits within: its review queue, ML-based identification step, and multi-stage confirmation workflow (Sections VIII, XII) are exactly the kind of background, asynchronous processes for which users increasingly expect live status rather than a manual refresh.

Event-driven programming (EDP) structures a system around discrete events rather than a linear sequence of instructions: independent event producers emit occurrences (a user tapping "submit," a model finishing inference, a moderator approving a record) that are placed on an event loop and dispatched to registered handlers, often through a publish/subscribe (Pub/Sub) channel that decouples the component raising an event from the components reacting to it. The pattern traces to the Observer design pattern, in which subjects notify a list of dependent observers of state changes without needing to know their concrete types [9], and it is formalized for modern distributed and web systems in the Reactive Manifesto, which frames responsive, resilient, elastic, and message-driven design as mutually reinforcing properties of a well-built reactive system [10]. For PhytoQuest, this framing reclassifies several already-specified behaviors — asynchronous ML inference (Section XII), submission-status notification (FR-13), and multi-stage review escalation — as event flows rather than blocking request/response calls, which motivates the architectural extension proposed in Section III.D.

Moving to an event-driven design does not eliminate complexity, however; it relocates it. When more than one community member or reviewer can act on the same sighting concurrently (Sections VIII.C–D), event ordering determines whether a late-arriving expert rejection correctly overrides an earlier community confirmation, or whether a race condition instead leaves a record in an inconsistent status. Asynchronous state also complicates the client: a submitted sighting can be in-flight to the ML service, awaiting community confirmation, and awaiting expert review simultaneously, and the interface must represent all three without implying a certainty the system has not yet earned (Section IX, Performance). These operational pain points — race conditions, event-ordering guarantees, and asynchronous state management — are the specific problems this paper's methodology (Section V) is designed to test.

II. PROBLEM STATEMENT

1. Researchers and students may lose substantial preliminary-research time locating viable medicinal plant specimens and assembling initial botanical information.
2. Static or difficult-to-navigate databases create friction when users need a mobile-friendly way to search, compare, and document plant information in the field.
3. Local plant records, images, and research instructions, or consumption advice.

These problems are not unique to botanical fieldwork tools; they are a specific instance of a broader architectural question. Purely synchronous, request-response designs do not uniformly outperform or underperform event-driven alternatives — the outcome depends on workload shape. An exploratory study by Cabane and Farias found that a monolithic, synchronous implementation of a test application was in fact more resource-

efficient and had better response times than an event-driven version of the same system [20], while a separate study of microservices found the opposite: event-driven internal communication was roughly 19% faster with a 34% lower error rate than a synchronous equivalent, at the cost of about 9% higher CPU usage [21]. A third comparison, of Node.js's event-driven runtime against a thread-based framework under load up to 64,000 virtual users, found Node.js faster on read-heavy operations but the thread-based alternative more reliable on complex writes at extreme concurrency [25]. Left unaddressed, PhytoQuest's current purely synchronous request/response design (Section X) risks the failure mode all three studies associate with the "wrong" architecture for a given workload: slow or blocking responses during concurrent field-session submissions, degraded reviewer throughput during high-volume identification periods, and — because status changes are not pushed to the client — users left uncertain whether a submission succeeded, was queued, or failed silently.

III. OBJECTIVE AND SCOPE

A. Objective

Develop and deploy a user-friendly, location-aware plant identification and information system that reduces the time spent locating and profiling medicinal flora for preliminary research activities.

B. In Scope

- Public plant search and access to basic plant profiles.
- Account-based scanning, submissions, history, favorites, and community confirmations.
- GPS-confirmed sighting submission, map display, reviewer workflow, and notifications.
- Image classification of flowers, leaves, stems, bark, fruits, and whole-plant views.
- Verified botanical, habitat, toxin, extract, and reference records with source labels.

C. Out of Scope

- Diagnosis, dosage, treatment recommendations, preparation instructions, or consumption advice.
- Chemical measurement or toxin quantification from a photograph; laboratory analysis remains necessary.
- Unreviewed publication of sensitive coordinates or private-property locations.

D. Proposed Solution Overview

To address the gaps in Section II under an event-driven paradigm, PhytoQuest's application layer will be extended with three coordinating mechanisms: (1) an event bus / backend queue that decouples ML inference and review-escalation processing from the request that triggers them, so a slow classification never blocks the submitting client; (2) client-side UI triggers that subscribe to status-change events (identification complete, review outcome, escalation required) instead of polling for updates; and (3) an event-driven API layer that emits a discrete event for each state transition in the sighting lifecycle (submitted, queued, identified, community-reviewed, expert-reviewed, published/rejected), so the React client and any future integrations can react to the same event stream. Expected outcomes are lower perceived latency while awaiting an ML result, higher throughput under concurrent field-session submissions, and improved usability from real-time status feedback in place of manual refreshing.

E. Contributions

- **Technical:** an event-driven extension to PhytoQuest's REST-based architecture (Section X) — an event bus, backend queue, and WebSocket-based notification channel layered onto the existing React–Laravel–Python stack — specified in Sections IV–V.
- **Empirical:** field-pilot performance data (response time, throughput, and concurrent-event-handling behavior) comparing the current synchronous flow against the proposed event-driven flow (Section XV).
- **Practical:** a reference design for small research and conservation teams who need real-time submission feedback without operating a large-scale messaging platform.
- **SDG alignment:** this contribution most directly advances SDG 9 (Industry, Innovation, and Infrastructure) by modernizing PhytoQuest's data and workflow infrastructure with resilient, message-driven components; the full SDG mapping appears in Section XVI.

IV. LITERATURE REVIEW

A. Foundations of Event-Driven Architecture

Event-driven architecture (EDA) rests on decades-old theoretical groundwork that current systems research is only now examining empirically. The Observer pattern [9] and the publish/subscribe formalization by Eugster et al. [12] both establish decoupling — in time, space, and synchronization — as the defining property of event-driven communication, while Luckham's complex event processing framework [11] extends this from simple notification to pattern detection across event streams. The Reactive Manifesto [10] later repackages these ideas as an industry-facing set of properties (responsive, resilient, elastic, message-driven), presenting message-driven design as a means to the other three ends rather than an end in itself. However, Lazzari and Farias argue that industry adoption of EDA has significantly outpaced empirical validation of its claimed benefits [13], and Bação and Guerreiro's review shows that even at the architectural level, consistency (data, process, and system-evolution) remains an open and only partially catalogued problem for event-driven systems [14]. Read together, these sources show a field with strong conceptual foundations but a thin empirical base connecting those foundations to measurable outcomes in specific applications.

Research gap: no reviewed source empirically validates the Observer/Pub-Sub/Reactive principles against a synchronous baseline for a small, single-team web application comparable in

scope to PhytoQuest — motivating this paper's applied comparison in Section V.

B. Event-Driven Programming in Web Applications

Within web applications specifically, the literature diverges on whether event-driven mechanisms are being adopted as fast as their conceptual appeal suggests. Sun et al. show that Node.js's asynchronous event-loop model, while foundational to server-side EDP, is also a persistent source of real bugs from mixed callback/promise APIs — motivating tooling built simply to reason about event ordering correctly [15]. On the transport side, Murley et al.'s large-scale crawl finds that WebSocket adoption across the top million websites has stagnated and that HTTP polling remains far more common than genuine push-based delivery, contradicting any assumption that real-time channels are now the default [16]. Front-end frameworks show a similar gap: Bai's Million.js demonstrates that React's virtual-DOM event/render model still leaves substantial performance on the table [17], and Zhang et al.'s interview study finds that even developers using modern CRDT libraries for real-time collaborative state are frequently confused about local-vs-collaborative event handling [19]. Kowalczyk and Szandala add a caveat from the single-page-application literature: client-rendered, event-driven architectures can match traditional multi-page SEO performance only with deliberate optimization, not by default [18]. Collectively, these studies suggest EDP's presence in production web stacks is real but uneven, and that naive adoption — without attention to event ordering, transport choice, and rendering strategy — does not automatically deliver the responsiveness EDP promises.

Research gap: existing studies benchmark EDP mechanisms individually (event loop, WebSockets, virtual DOM) rather than as an integrated feature set added to an existing synchronous REST application, which is precisely the extension this paper proposes for PhytoQuest (Section III.D).

C. Performance & Scalability Studies (Request-Response vs. Event-Driven)

The performance literature directly undercuts a simple "event-driven is always faster" narrative. Cabane and Farias found that a monolithic, synchronous implementation used computational resources more efficiently and had better response times than an equivalent event-driven version of the same application [20]. Rahmatulloh et al., testing microservices rather than a monolith, found the opposite: event-driven internal communication was about 19% faster with a 34% lower error rate than a synchronous equivalent, but at roughly 9% higher CPU cost — showing the trade-off is resource-shaped, not universal [21]. At larger scale, Grambow et al.'s benchmarking framework and Ramadan et al.'s Kafka/RabbitMQ comparison both show that the "best" event-driven configuration depends heavily on workload shape: FaaS platforms vary by provider and application profile [22], and RabbitMQ outperforms Kafka on low-latency individual messages while Kafka wins on sustained high-throughput load [24]. Shyam Mohan and Goswami's Node.js-vs-Spring-Boot study sharpens this further: Node.js's event-driven model won on read-heavy operations and CPU efficiency, but the thread-based alternative handled complex writes and error rates better at extreme concurrency [25]. Arteca et al. show a middle path — statically reordering async operations can recover meaningful throughput (up to ~54% in their benchmarks) without changing the underlying architecture at all [23]. The consistent theme is that request-response and event-driven approaches trade different resources (CPU, latency, error rate, throughput) under different load shapes, and no single architecture dominates across all of them.

Research gap: none of these studies isolate the specific workload shape PhytoQuest expects — bursty, field-session-driven submissions with a slow ML-inference step in the critical path — leaving open whether an event-driven extension actually helps this application's real bottleneck (Section V proposes measuring this directly).

D. Testing and Debugging Challenges in Asynchronous Systems

Where the performance literature disagrees on which architecture wins, the testing literature agrees that asynchrony itself is a primary cost. Parry et al.'s survey and Hashemi et al.'s empirical study both identify concurrency and async-wait behavior as leading causes of flaky tests, with Hashemi et al. showing this dominates in JavaScript specifically, unlike the test-order dependencies that dominate in Java [26], [27]. Detection is harder than it looks: Alshammari et al. reran test suites 10,000 times each and still found many flaky tests undetected, motivating prediction models that do not rely on repeated execution at all [28]. At the code level, Turcotte et al.'s DrAsync catalogs eight recurring async/promise anti-patterns that are both common (thousands of instances found across only 20 repositories) and often simultaneously a correctness and a performance problem [29]. Endo and Møller's NodeRacer and its successor target the specific failure mode this paper cares about most — event races caused by ambiguous callback ordering — and show that delay-injection techniques can expose these races with fewer executions than naive repeated testing [30], [31]. Together, these sources show that async-related defects are common, expensive to detect through repetition alone, and require dedicated tools rather than conventional test practice.

Research gap: no reviewed source proposes a concrete test plan for event-ordering and race conditions in a student-built, non-Node.js web application (PhytoQuest uses Laravel/React/Python, not a Node.js backend), which is the gap this paper's black-box test scenarios (Section XIV) are designed to fill.

V. METHODOLOGY

A. Research Track

This study follows Track B: Software Engineering Methodology, using the Software Development Life Cycle (SDLC) as the organizing frame for design, data collection, and analysis (Sections V.B–V.D), blended with a small experimental comparison between the current synchronous flow and the proposed event-driven flow during the pilot (Section XV).

B. Design

The design phase produces: (1) an updated architecture diagram extending Section X's three-service architecture with an event bus/backend queue and a WebSocket notification channel; (2) event-flow diagrams for the two flows in Section XI (Search and Map Flow; Sighting Submission Flow), re-expressed as event sequences (submitted → queued → identified → reviewed → published); and (3) a recorded tech-stack version baseline (client framework, backend framework, ML runtime, and message-queue/broker versions), captured at the start of implementation so later performance results are reproducible. *[Team: record your exact versions here — e.g., React, Laravel, and Python versions, plus the chosen queue/broker — before implementation begins.]*

C. Data Collection / Scoping

Requirements were gathered from the functional and nonfunctional specifications already defined in Sections VIII–IX, supplemented with performance-log metrics captured from the running application (per-request timestamps at submission, queue-enqueue, ML-inference-complete, and notification-delivery events) and, if usability testing is conducted, a convenience sample of pilot participants drawn from the partner campus/site named in Section XV (target n and inclusion criteria to be finalized with the course instructor).

D. Data Analysis

Log data will be analyzed by event stage (queue wait time, inference time, notification delivery time) to compute response time (ms), throughput (requests/events per second), and concurrent-event-handling behavior under simulated multi-user load. Usability will be evaluated with the System Usability Scale (SUS) and, where applicable, ISO/IEC 25010 quality characteristics (performance efficiency, reliability, usability) as a secondary framework for interpreting the pilot metrics already defined in Section XV.

E. Rigor & Ethics

Validity and reliability will be supported by pilot-testing the event-driven flow on a non-production dataset before the campus pilot, and by repeating load trials at least three times per condition (synchronous vs. event-driven) to check for consistent results. Ethical handling follows the data-governance boundaries already defined in Section XIII: sensitive-location generalization, role-based access, and audit logging apply unchanged to event payloads. Any human-subjects usability testing will use informed consent, voluntary participation, and anonymized session identifiers, consistent with the institution's research ethics requirements.

VI. STARR EXPLANATION

A. Situation

Students and field researchers need plant specimens and credible basic information before laboratory investigation, yet discovery and initial data gathering can be slow and fragmented.

B. Task

Provide a research-oriented system that links field discovery, image-based candidate identification, reviewed location evidence, and traceable botanical information while avoiding clinical or medical advice.

C. Action

PhytoQuest captures GPS-confirmed submissions, applies expert review and community confirmation, processes plant-part images through a Python model, and retrieves only curated profile data from the Laravel-managed database. The React interface presents these functions in a mobile-first workflow.

D. Result

Users can move from a plant search to a defensible field record and a traceable information profile with fewer disconnected tools. Uncertain identifications remain explicitly uncertain, and sensitive locations remain protected.

VII. USERS, ROLES, AND ACCESS

Role	Access and responsibility
Guest	Search plants; view basic profiles and map areas allowed for public display.
Registered user	Scan plants; save history and favorites; add and confirm sightings.
Expert reviewer	Assess identity claims, location accuracy, references, and sensitive-record status.
Admin	Manage accounts, roles, data sources, taxonomies, audit records, and final publication controls.

VIII. FUNCTIONAL REQUIREMENTS

A. Discovery and Profiles

FR-01. The system shall allow guests to search by common name and scientific name.

FR-02. The system shall provide filters for habitat, plant part, location, toxin, extract, and category.

FR-03. A plant profile shall show the accepted scientific name, common names, physical features, habitat, verified occurrence records, known toxins, documented extract types, and research references.

FR-04. Each information item shall show its source label: trusted database, research paper, or expert-reviewed submission.

B. Scanning and Identification

FR-05. Authenticated users shall capture or upload images tagged as flower, leaf, stem, bark, fruit, or whole plant.

FR-06. The Python service shall return ranked candidate species and confidence scores, rather than an unqualified confirmation.

FR-07. When confidence is low, the application shall request additional images, present possible matches, and allow escalation to expert review.

C. Location Records

FR-08. The application shall request GPS permission and require the user to confirm or adjust the recorded pin before submission.

FR-09. A sighting submission shall contain coordinates, date, at least one image, plant-part tags, public/private/protected status, and optional notes.

FR-10. Any registered user may propose a new record; it shall remain pending until expert review.

FR-11. Community confirmations shall contribute evidence but shall not replace expert approval for sensitive records.

D. Account Features

FR-12. Authenticated users shall retain searches, scans, submissions, confirmations, and favorites in their history.

FR-13. The system shall notify a submitting user when a record is approved, rejected, or requires correction.

FR-14. The first release shall allow selected profiles to be saved for offline viewing; it shall not promise offline maps or offline classification.

IX. NONFUNCTIONAL REQUIREMENTS

- **Usability:** the interface shall be responsive and optimized for phone use during fieldwork.
- **Traceability:** profile claims and location records shall retain source, contributor or reviewer, status, date, and change history.
- **Security:** authenticated actions shall use role-based authorization; submitted images and coordinates shall be protected in transit and at rest.
- **Privacy:** public map precision shall follow conservation status and location-ownership permission.
- **Performance:** ordinary search and profile requests should return promptly on a standard mobile connection; image analysis shall show processing status rather than imply instant certainty.
- **Maintainability:** the React client, Laravel API, and Python model service shall remain separately deployable through documented interfaces.

X. PROPOSED ARCHITECTURE

The system uses a three-service architecture with a separate data layer. React JS is responsible for the browser-side experience: account screens, search, maps, camera and file input, GPS consent, submission forms, and result presentation. Laravel PHP exposes authenticated REST endpoints, applies validation and role rules, manages review queues, and retrieves verified profile content. Python hosts the image model and returns ranked candidate species with confidence values. A relational database stores authoritative application records; object storage holds submitted images; and an audit log records moderation actions.

A. Component Responsibilities

Component	Primary responsibility
React JS client	Mobile-first UI, camera/GPS permission flow, maps, search, forms, and result states.
Laravel PHP API	Authentication, roles, user profiles, sightings, reviews, history, notifications, and audit logs.
Python ML service	Image preprocessing, species candidate ranking, confidence scoring, and model-version reporting.
Data layer	Relational records, controlled vocabulary, references, conservation rules, image storage, and backups.

B. Key Data Entities

- **Plant:** accepted taxon, synonyms, common names, botanical features, habitat, and category.
- **Profile evidence:** toxin or extract statement, source type, citation, reviewer status, and date reviewed.
- **Sighting:** taxon candidate, location, observed date, images, plant-part tags, visibility class, ownership status, and review state.
- **User and role:** account identity, permissions, submission history, reviewer credentials, and audit events.
- **Model inference:** image set, model version, ranked candidates, confidence, and escalation outcome.

XI. DATA FLOW AND VERIFICATION

A. Search and Map Flow

1. The user searches or filters a plant record.
2. Laravel retrieves profile content and only location permitted for the user and record class.
3. React renders exact pins for eligible common-area records and generalized areas for sensitive, unverified, or private records.

B. Sighting Submission Flow

1. The user signs in, grants GPS permission, and adjusts the map pin.
2. The user adds images, plant-part tags, date, location status, and notes.
3. Laravel validates the request, stores the pending record, and sends images to the Python service when identification is requested.
4. The system records candidate species and confidence scores, turning the prediction into a confirmed taxon.
5. Community confirmations and expert review contribute to a final status: pending, needs correction, trusted, rejected, or sensitive-restricted.

C. Location Protection Rule

Common plants in public areas may show exact coordinates after approval. Rare, endangered, vulnerable, or privately located plants shall show only a generalized area. The conservation class is first obtained from official conservation sources and may be amended by qualified expert review when data are absent, outdated, or disputed. Generalizing sensitive occurrence data follows established biodiversity-data practice [2].

XII. MACHINE-LEARNING WORKFLOW

PhytoQuest uses transfer learning: a pre-trained vision model is fine-tuned with licensed, labeled images relevant to the pilot region and target taxa. This approach is more realistic than training an entire model from scratch and supports future adaptation with locally reviewed samples. Plant computer-vision research also shows that image-based classification remains a recognition task with known limitations, particularly across visually similar species and variable field conditions [5], [6].

1. Collect candidate training data from licensed datasets, trusted sources, and expert-approved submissions; retain source and license metadata.
2. Normalize images, preserve plant-part labels, and split data by observation or collection event to reduce leakage between training and testing.
3. Fine-tune and evaluate a candidate model with per-species precision, recall, top-k accuracy, confusion matrices, and low-confidence behavior.
4. Deploy a versioned model that returns ranked matches and confidence values through a documented API.
5. Retrain only with expert-approved images after a data-quality review; do not automatically learn from every user submission.

The model identifies a likely species from visual evidence. It does not infer the chemical content, concentration, toxicity level, or safety of a photographed specimen. Profile information is retrieved from reviewed database records after candidate matching.

XIII. DATA GOVERNANCE AND SAFETY BOUNDARIES

- **No medical advice:** PhytoQuest shall not diagnose conditions, recommend treatment, prescribe dosage, recommend preparation, or tell users to consume any plant.
- **Evidence separation:** traditional use, laboratory evidence, animal evidence, human studies, and database records shall not be presented as equivalent claims.
- **Source discipline:** plant data imports and occurrence records shall preserve the provider, access date, license, identifier, and citation. Darwin Core provides a suitable vocabulary for biodiversity occurrence data [3].
- **Reference review:** titles, authors, publication year, source, and DOI or stable URL shall be stored for each research reference. Imported biodiversity data must retain the appropriate citation and attribution [4].
- **Conservation and property:** sensitive coordinates are generalized; private locations are hidden unless the owner gives permission; access for verified research can be controlled by administrators.
- **Human oversight:** qualified botanists, pharmacists, researchers, and approved faculty review disputed identities, evidence statements, and sensitive records.

These boundaries are essential because herbal information can involve safety issues and interactions with other medicines. The system therefore supports research and information retrieval rather than personal medical decisions [1].

XIV. INITIAL WEB SYSTEM FEATURES AND TEST PLAN

A. Core Event-Driven Features

- **Real-Time Notifications / Streaming:** extend FR-13 so status-change events (queued, identified, community-confirmed, expert-approved/rejected) are pushed to the submitting user's client over a WebSocket (or equivalent) channel rather than requiring a manual refresh.
- **Asynchronous Event Handlers & State Management:** attach event listeners to user actions not yet covered by a discrete requirement — inline validation as a sighting form is completed (coordinates, required photo, plant-part tags) and drag-and-drop reordering of multi-part images before submission — so the client reflects validity and ordering instantly instead of only at submit time.
- **Background Event Processing:** route ML inference and review escalation through a backend event queue with named subscribers (ML worker, notification service, audit logger) instead of direct synchronous calls, so a slow inference cannot block the submitting request (extends Sections X and XII).

B. Feature Metrics & Test Scenarios

- **Metrics:** response time (ms) per event stage, throughput (events/sec) under concurrent field-session submissions, concurrent-event-handling correctness (no lost or duplicated status transitions), and a running defect/bug log tagged by event stage.
- **SUS:** administer the 10-item System Usability Scale to pilot participants (Section XV) after they complete a full submission-to-notification cycle.
- Initial black-box test cases:

1. Submit a sighting with a simulated ML delay → confirm the client shows a distinct “processing” state and later updates via the event channel without a manual refresh.
2. Two community members confirm the same sighting concurrently → confirm only one final status is recorded and no duplicate notification is sent.
3. An expert rejects a sighting after a community confirmation has already been recorded → confirm event

ordering resolves to "rejected," not a stale "confirmed" state.

4. Submit sightings from three simulated concurrent users → confirm all three receive correct, independent status notifications with no cross-delivery.

XV. PILOT, EVALUATION, AND SUCCESS MEASURES

The first deployment shall be limited to one campus or selected partner locations. The restricted pilot makes it possible to verify location ownership, establish reviewer capacity, and measure performance before broader deployment.

Measure	Pilot evidence
Field efficiency	Median time to locate a target record and complete a usable sighting submission.
Identification quality	Top-k accuracy, low-confidence escalation rate, and reviewer agreement for target taxa.
Record quality	Share of submissions complete at first review; approval, correction, and rejection rates.
Usability	Task-completion rate and structured feedback from pharmacy, botany, and engineering participants.
Safety and privacy	Number of visible sensitive locations, permission exceptions, or unsupported medical claims (target: zero).

XVI. SDG ALIGNMENT

- SDG 3 Good Health and Well-Being: supports earlier-stage, evidence-traceable medicinal-plant research without substituting for medical care.
- SDG 4 Quality Education: gives students a structured field-research tool for self-directed learning and documented observation.
- SDG 9 Industry, Innovation, and Infrastructure: modernizes botanical information workflows through geospatial records, review processes, and computer vision.
- SDG 15 Life on Land: documents plant occurrence while protecting sensitive locations and encouraging responsible conservation-aware use of data.

XVII. CONCLUSION

PhytoQuest is a botanical research support system designed to shorten the path from field discovery to organized, source-backed plant information. Its value comes from connecting location evidence, multi-part image scanning, careful review, and a focused plant profile not from replacing expert judgment or medical practice. By starting with a controlled pilot and explicit evidence, privacy, and safety boundaries, the project can produce useful research infrastructure while remaining technically and ethically realistic.

REFERENCES

[1] World Health Organization, "Key technical issues of herbal medicines with reference to interaction with other medicines," 2021. [Online]. Available: https://www.who.int/publications/b/55639. [Accessed: Jul. 24, 2026].

[2] A. D. Chapman, Current Best Practices for Generalizing Sensitive Species Occurrence Data. Copenhagen, Denmark: GBIF Secretariat, 2020, doi: 10.15468/doc-5jp4-5g10.

[3] Biodiversity Information Standards (TDWG), "Darwin Core," [Online]. Available: https://dwc.tdwg.org/. [Accessed: Jul. 24, 2026].

[4] Global Biodiversity Information Facility, "Citation guidelines," [Online]. Available: https://www.gbif.org/citation-guidelines. [Accessed: Jul. 24, 2026].

[5] J. Wäldchen and P. Mäder, "Plant species identification using computer vision techniques: A systematic literature review," Arch. Comput. Methods Eng., vol. 25, pp. 507–543, 2018, doi: 10.1007/s11831-016-9206-z.

[6] S. Wäldchen, P. Mäder, and M. Rzanny, "Automated plant species identification—Trends and future directions," PLoS Comput. Biol., vol. 14, no. 4, e1005993, 2018, doi: 10.1371/journal.pcbi.1005993.

[7] R. Govaerts et al., "The World Checklist of Vascular Plants, a continuously updated resource for exploring global plant diversity," Sci. Data, vol. 8, art. 215, 2021, doi: 10.1038/s41597-021-00997-6.

[8] Royal Botanic Gardens, Kew, "Plants of the World Online," [Online]. Available: https://powo.science.kew.org. [Accessed: Jul. 24, 2026].

[9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, 1st ed. Boston, MA: Addison-Wesley Professional, 1994.

[10] J. Bonér, D. Farley, R. Kuhn, and M. Thompson, "The Reactive Manifesto," v2.0, Sep. 16, 2014. [Online]. Available: https://www.reactivemanifesto.org/.

[11] D. Luckham, The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems, 1st ed. Boston, MA: Addison-Wesley Professional, 2002.

[12] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," ACM Computing Surveys, vol. 35, no. 2, pp. 114–131, Jun. 2003, doi: 10.1145/857076.857078.

[13] L. Lazzari and K. Farias, "Uncovering the hidden potential of event-driven architecture: A research agenda," arXiv:2308.05270, Aug. 2023. [Preprint].

[14] J. Bação and S. Guerreiro, "Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution," Computing, vol. 108, no. 7, 2026, doi: 10.1007/s00607-026-01701-5.

[15] H. Sun, D. Bonetta, F. Schiavio, and W. Binder, "Reasoning about the Node.js event loop using async graphs," in Proc. 2019 IEEE/ACM Int. Symp. Code Generation and Optimization (CGO), 2019, pp. 61–72, doi: 10.1109/CGO.2019.8661173.

[16] P. Murley, Z. Ma, J. Mason, M. Bailey, and A. Kharraz, "WebSocket adoption and the landscape of the real-time web," in Proc. Web Conf. 2021 (WWW '21), 2021, pp. 1192–1203, doi: 10.1145/3442381.3450063.

[17] A. Bai, "Million.js: A fast compiler-augmented virtual DOM for the web," in Proc. 38th ACM/SIGAPP Symp. Applied Computing (SAC '23), 2023, doi: 10.1145/3555776.3577683.

[18] K. Kowalczyk and T. Szandala, "Enhancing SEO in single-page web applications in contrast with multi-page applications," IEEE

Access, vol. 12, pp. 11597–11614, 2024, doi: 10.1109/ACCESS.2024.3355740.

[19] Y. Zhang, M. Weidner, and H. Miller, “Programmer experience when using CRDTs to build collaborative webapps: Initial insights,” in Proc. 13th Annu. Workshop Intersection of PL and HCI (PLATEAU 2023), 2023, doi: 10.1184/R1/22277341.v1.

[20] H. Cabane and K. Farias, “On the impact of event-driven architecture on performance: An exploratory study,” Future Generation Computer Systems, vol. 153, pp. 52–69, 2024, doi: 10.1016/j.future.2023.10.021.

[21] A. Rahmatulloh, F. Nugraha, R. Gunawan, and I. Darmawan, “Event-driven architecture to improve performance and scalability in microservices-based systems,” in Proc. 2022 Int. Conf. Advancement in Data Science, E-Learning and Information Systems (ICADEIS), 2022, doi: 10.1109/ICADEIS56544.2022.10037390.

[22] M. Grambow, T. Pfandzelter, L. Burchard, C. Schubert, M. X. Zhao, and D. Bermbach, “BeFaaS: An application-centric benchmarking framework for FaaS platforms,” in Proc. 2021 IEEE Int. Conf. Cloud Engineering (IC2E), 2021, pp. 1–8.

[23] E. Arteca, F. Tip, and M. Schäfer, “Enabling additional parallelism in asynchronous JavaScript applications,” in Proc. 35th Eur. Conf. Object-Oriented Programming (ECOOP 2021), 2021, pp. 7:1–7:28, doi: 10.4230/LIPIcs.ECOOP.2021.7.

[24] M. R. Ramadan, A. M. Umam, A. Asmani, and R. M. Ijtihadie, “A comparative study evaluation of Kafka and RabbitMQ: Performance, scalability and stress test in distributed messaging systems,” JUTI: Jurnal Ilmiah Teknologi Informasi, vol. 24, no. 1, pp. 60–86, 2026, doi: 10.12962/j24068535.v24i1.a1345.

[25] J. S. Shyam Mohan and K. Goswami, “Performance analysis and comparison of Node.js and Java Spring Boot in implementation of RESTful applications,” Software: Practice and Experience, vol. 55, no. 7, pp. 1209–1233, 2025, doi: 10.1002/spe.3418.

[26] N. Hashemi, A. Tahir, and S. Rasheed, “An empirical study of flaky tests in JavaScript,” in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2022, pp. 24–34, doi: 10.1109/ICSME55016.2022.00011.

[27] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “A survey of flaky tests,” ACM Trans. Softw. Eng. Methodol., vol. 31, no. 1, pp. 1–74, 2021, doi: 10.1145/3476105.

[28] A. Alshammari, C. Morris, M. Hilton, and J. Bell, “FlakeFlagger: Predicting flakiness without rerunning tests,” in Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE), 2021, pp. 1572–1584, doi: 10.1109/ICSE43902.2021.00140.

[29] A. Turcotte, M. D. Shah, M. W. Aldrich, and F. Tip, “DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript,” in Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE), 2022, pp. 774–785, doi: 10.1145/3510003.3510097.

[30] A. T. Endo and A. Møller, “NodeRacer: Event race detection for Node.js applications,” in Proc. IEEE Int. Conf. Software Testing, Verification and Validation (ICST), 2020, pp. 120–130, doi: 10.1109/ICST46399.2020.00022.

[31] A. T. Endo and A. Møller, “Event race detection for Node.js using delay injections,” in Proc. Eur. Conf. Object-Oriented Programming (ECOOP), 2025, pp. 9:1–9:28, doi: 10.4230/LIPIcs.ECOOP.2025.9.